

Documentação FSM

Castelo

1-Visão Geral do Cenário

O sistema simula a rotina contínua de atividades em um cenário medieval por meio de uma Finite State Machine estruturada com o padrão State. O motor de execução gerencia uma lista de Character composta por um Blacksmith e um Guard ao longo de um numero determinado de iterações controladas por um laço de repetição. A cada ciclo, o método update() de cada agente delega a lógica operacional para classes de estado independentes (que herdam de AbstractState) através do método execute(), acionando métodos dos próprios personagens para modificar seus atributos internos. Quando os limiares de transição são atingidos, o método setState() coordena as mudanças: executa o método leave() do estado que se encerra (emitindo avisos de conclusão e zerando marcadores de tempo) e invoca imediatamente o método enter() do novo estado (exibindo mensagens de início de ação e reiniciando variáveis de produção).

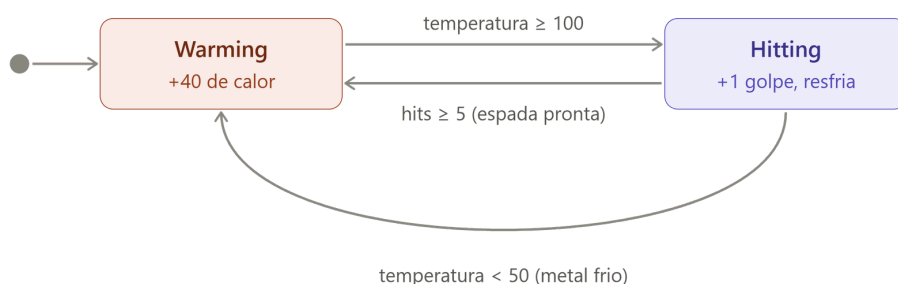
2-Visão Geral dos Agentes

Ferreiro (Blacksmith)

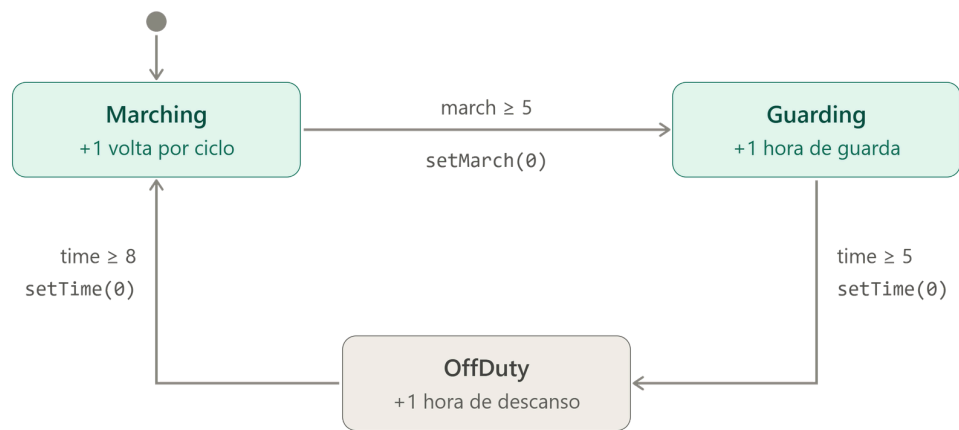
O Ferreiro é o agente encarregado da forja de espadas, controlando seu fluxo de trabalho através dos atributos de temperatura (temperature) e golpes (hits), tendo o aquecimento (Warming) como estado inicial. Durante a execução de Warming, o método parametrizado warmingMetal(40) eleva a temperatura em 40 unidades por ciclo até atingir ou superar 100 graus; nesse ponto, o estado é encerrado com o aviso "Metal quente!" (leave) e o agente transita para o estado Hitting anunciando "Indo para a bigorna" (enter). Em Hitting, cada ciclo incrementa os golpes em 1 unidade via hittingMetal(1) e reduz o calor em um valor aleatório de 0 a 29 graus via coolingMetal(30). Se a temperatura cair abaixo de 50 graus antes do término da peça, a saída de Hitting alerta "O metal esfriou muito!" e a entrada de Warming exibe "De volta ao fogo", preservando os golpes acumulados; porém, ao alcançar 5 marteladas (hits >= 5), a saída de Hitting declara "Espada pronta!" e a entrada subsequente em Warming imprime "Fazendo a próxima", zerando os atributos hits e temperature para iniciar uma nova espada.

3-Diagrama de estados

Ferreiro



Guarda



4 -Tabela de regras

Ferreiro (Blacksmith)

Estado inicial: Warming (temperatura = 0, golpes = 0)

Estado	Entrada (enter)	Execução (execute)	Saída (leave)	Transições
Warming	Se hits >= 5: imprime "Fazendo a próxima" e zera hits e temperature. Senão, se temperature < 50: imprime "De volta ao fogo".	Aquece o metal em +40 e imprime "Aquecendo metal (Calor: X)".	Imprime "Metal quente!"	→ Hitting quando temperature >= 100
Hitting	Imprime "Indo para a bigorna".	hits +1; resfria o metal em valor aleatório de 0 a 29; imprime "CLANG" X/5 (Temperatura caindo para Y)".	Se hits >= 5: "Espada pronta!". Senão, se temperature < 50: "O metal esfriou muito!".	→ Warming quando hits >= 5 → Warming quando temperature < 50

Guarda (Guard)

Estado inicial: Marching (march = 0, time = 0)

Estado	Entrada (enter)	Execução (execute)	Saída (leave)	Transições
Marching	Imprime "Hora de marchar!"	march +1; imprime "Marchando ao redor do castelo (Voltas: X)".	Imprime "Patrulha completa!"	→ Guarding quando march >= 5 (depois zera march)
Guarding	Imprime "Hora de cuidar da entrada!"	time +1; imprime "Guardando a entrada (Horas: X)".	Imprime "Fim de turno" e zera time.	→ OffDuty quando time >= 5
OffDuty	Imprime "Hora de descansar!!"	time +1; imprime "Fora do turno (Horas: X)".	Imprime "Descanso concluído!" e zera time.	→ Marching quando time >= 8

5-Descrição das variáveis:

Guard

- march:**
Variável inteira (int, privada) que começa com o valor 0. Durante o estado Marching, ela aumenta em +1 a cada rodada por meio do método marchAround(1), variando no intervalo de 0 a 5.
- time:**
Variável inteira (int, privada) que começa com o valor 0 e marca a passagem das horas nos estados Guarding e OffDuty. Ela aumenta em +1 por rodada através do método passTime(1), variando de 0 a 5 durante a vigilância (Guarding) e de 0 a 8 durante o descanso (OffDuty).

- **state:**
Variável do tipo State< Guard > (privada) que guarda o estado atual do personagem e começa definida como new Marching(this).

Blacksmith

- **temperature:**
Variável inteira (int, privada) que começa em 0 e representa o calor do metal. No estado Warming, ela sobe +40 graus por rodada através do método warmingMetal(40); já no estado Hitting, ela perde entre 0 e 29 graus de forma aleatória por rodada através do método coolingMetal(30).
- **hits:**
Variável inteira (int, privada) que começa em 0 e conta as marteladas dadas na espada. Durante o estado Hitting, ela aumenta em +1 por rodada pelo método hittingMetal(1).
- **random e randomDrop:**
random é um gerador de números aleatórios (new Random()) da classe Blacksmith, usado dentro do método coolingMetal(int temperature) para definir o valor da variável local randomDrop (int). Como o estado Hitting chama coolingMetal(30), a variável randomDrop sorteia um número entre 0 e 29 a cada rodada usando random.nextInt(temperature). Ela não possui um limite de transição próprio, servindo apenas para descontar esse valor sorteado da variável temperature.
- **state:**
Variável do tipo State (privada) que começa definida como new Warming(this) e armazena o estado atual do ferreiro. Ela alterna entre os estados Warming e Hitting por meio do método setState(), que chama o leave() do estado anterior, salva o novo estado na variável e executa o seu método enter().

6 - Estrutura do código:

O projeto é dividido em 11 arquivos .java, organizados entre a estrutura base da máquina de estados, as classes dos personagens e as classes que representam cada estado:

1. Base e Controle

- **FSM.java:** Classe principal que contém o método main e inicia o programa. Ela cria a lista de personagens com o Ferreiro (Blacksmith) e o Guarda (Guard) e executa um laço de 40 rodadas (contador <= 40), chamando o método update() de cada agente a cada repetição.
- **Character.java:** Interface que define o padrão obrigatório para os personagens do projeto. Ela garante que tanto o Ferreiro quanto o Guarda tenham os métodos update() para rodar o turno, setState() para trocar de estado e printStats() para mostrar informações na tela.
- **State.java:** Interface que define o formato padrão de todos os estados do sistema, exigindo os métodos de entrada (enter()), execução da ação (execute()) e saída (leave()).
- **AbstractState.java:** Classe abstrata que implementa a interface State e serve de modelo para os estados específicos. Ela guarda a referência do personagem dono daquele estado e permite acessá-lo por meio do método getCharacter().

2. Classes do Ferreiro (Blacksmith)

- **Blacksmith.java:** Classe do agente Ferreiro, responsável por guardar a temperatura do metal (temperature) e a contagem de marteladas (hits), começando no estado Warming. Ela reúne os métodos que alteram esses valores (warmingMetal, coolingMetal e hittingMetal), além de controlar a execução do turno em update() e a transição entre estados em setState().
- **Warming.java:** Estado responsável por aquecer o metal, somando 40 graus por rodada

através de `warmingMetal(40)` até atingir ou passar de 100 graus e mudar para `Hitting`. Ao iniciar (`enter()`), verifica se uma espada acabou de ficar pronta (`hits >= 5`) para zerar os golpes e a temperatura, ou se o retorno ocorreu porque o metal esfriou (`temperature < 50`); ao encerrar (`leave()`), avisa que o metal está quente.

- **Hitting.java:** Estado em que o ferreiro trabalha na bigorna, somando 1 golpe via `hittingMetal(1)` e perdendo de 0 a 29 graus aleatoriamente via `coolingMetal(30)` a cada rodada. Ele avisa a ida para a bigorna em `enter()`, verifica se a espada chegou a 5 golpes (`hits >= 5`) ou se a temperatura caiu abaixo de 50 graus (`temperature < 50`) para retornar ao estado `Warming`, e exibe a mensagem correspondente ao motivo da saída em `leave()`.

3. Classes do Guarda (Guard)

- **Guard.java:** Classe do agente Guarda, que armazena a quantidade de voltas (`march`) e o tempo decorrido (`time`), tendo `Marching` como estado inicial. Possui os métodos para avançar a patrulha (`marchAround`) e as horas (`passTime`), além de gerenciar o ciclo de cada rodada em `update()` e as trocas de comportamento em `setState()`.
- **Marching.java:** Estado de patrulha em que o guarda caminha ao redor do castelo somando 1 volta por rodada via `marchAround(1)`. Exibe o aviso de início de marcha em `enter()`, informa a conclusão da patrulha em `leave()` e, quando atinge 5 voltas (`march >= 5`), muda para o estado `Guarding` e zera o contador de voltas (`setMarch(0)`).
- **Guarding.java:** Estado em que o guarda vigia a entrada do castelo, somando 1 hora por rodada através do método `passTime(1)`. Anuncia o início da vigilância em `enter()`, transfere o guarda para o estado de descanso (`OffDuty`) ao alcançar 5 horas (`time >= 5`) e usa o método `leave()` para avisar o fim do turno e zerar as horas (`setTime(0)`).
- **OffDuty.java:** Estado de descanso fora de turno, onde o guarda também contabiliza 1 hora por rodada usando `passTime(1)`. Avisa o início do descanso em `enter()`, retorna para o estado `Marching` ao completar 8 horas (`time >= 8`) e utiliza o método `leave()` para informar que o descanso terminou e zerar o relógio (`setTime(0)`).

7 - Resultados esperados

Ferreiro (Blacksmith)

1.De Warming para Hitting (temperature > 100)

Ferreiro

Aquecendo metal (Calor: 120)

Metal quente!

Indo para a bigorna

=====

2.De Hitting para Warming (hits >= 5)

Ferreiro

***CLANG* 5/5 (Temperatura caindo para 58)**

Espada pronta!

Fazendo a próxima

=====

3.De Hitting para Warming (temperature < 50)

Ferreiro

***CLANG* 2/5 (Temperatura caindo para 43)**

O metal esfriou muito!

De volta ao fogo

=====

Guarda (Guard)

1.De Marching para Guarding (march >= 5)

Guarda

Marchando ao redor do castelo(Voltas: 5)

Patrulha completa!

Hora de cuidar da entrada!

=====

2.De Guarding para OffDuty (time >= 5)

Guarda

Guardando a entrada (Horas: 5)

Fim de turno

Hora de descansar!!

=====

3.De OffDuty para Marching (time >= 8)

Guarda

Fora do turno (Horas: 8)

Descanso concluído!

Hora de marchar!

=====